SOURCE-LEVEL GUIDE • TINTINWEB/PI-SUBAGENTS

pi-subagents 源码导读

pi-subagents 把 Claude Code 风格的子 agent 和脚本化工作流带到 pi。这份导读讲清它怎么派生、排队、通知、隔离子 agent，工作流沙箱为什么这样设计，以及想贡献或定制时该看哪里。

tintinweb/pi-subagents @ 50460b5

怎么读这份导读

这份导读写给想读懂、贡献或基于 pi-subagents 做定制的工程师。读完你应该能说清：一次 `Agent` 工具调用怎么变成一个独立会话；完成通知为什么不会打断父 agent；工作流脚本为什么禁用 `Date.now()`；新功能该放在哪个文件。

全文顺序：

1. **全景**：问题、目标、限制、总图。
2. **agent 注册表**：内置、自定义、包内 agent 怎么合并。
3. **生命周期管理**：两个并发池、排队、状态。
4. **运行器**：怎么构造一个隔离的子会话。
5. **完成通知**：outbox 和 join。
6. **隔离与嵌套**：worktree、嵌套 agent、作用域。
7. **工作流**：脚本沙箱、确定性、journal 重放、gate。
8. **端到端**：一次后台 `Agent` 调用。
9. **许可证、贡献与定制、源码索引**。

> **要点 · 固定版本**
>
> `tintinweb/pi-subagents` @ `50460b5`（2026-11-04，版本 `0.23.1`）。需要 pi 1.1.0 以上。我没有运行过这个扩展，也没有跑过测试，内容来自阅读源码和文档。

全景：问题、目标、限制

它解决什么

pi 是一个可扩展的编码 agent，但核心不带子 agent。这个扩展补上这一层，并且刻意和 Claude Code 保持一样的工具名、调用约定和 UI（项目哲学）。

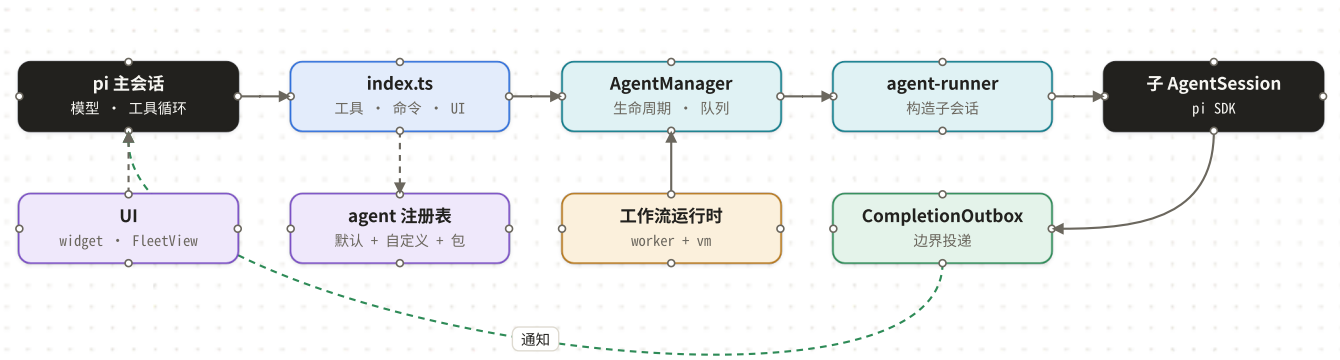
环节	问题	解决办法
上下文污染	主 agent 自己探索代码，上下文很快耗尽	子 agent 在独立会话里跑，只把结果带回来
串行等待	模型一次只能发一批工具调用	后台 agent + 完成通知
动态扇出	工具调用批次不能循环，也不能根据运行时发现的列表分支	`SubagentWorkflow` 脚本
并行写冲突	多个 agent 改同一棵树会互相覆盖	git worktree 隔离
验证靠嘴	“修好了”只是模型说的	工作流的 `gate`：跑命令，失败就算 agent 失败

要求 / 目标 / 限制

项目	内容
核心目标	让子 agent 在 pi 里感觉是原生的，和 Claude Code 行为一致
功能要求	后台/前台、恢复、steering、自定义 agent、工作流、调度、跨扩展 RPC
规模压力	工作流单次最多 1000 个 agent（`WORKFLOW_AGENT_CAP`）；TUI 不能被卡住
外部限制	运行在 pi 进程内；依赖 pi 的会话、事件和工具 API；peer 依赖 `>=1.1.0`
安全边界	和运行 pi 的用户同一信任边界，**不是沙箱**（`SECURITY.md`）
关键手段	独立 `AgentSession`；两个并发池；outbox；vm + worker；journal 重放
许可证	MIT

总图

扩展在 pi 里的位置



深色来自 pi，不在这个仓库里。所有东西都在**同一个 Node 进程**内运行，只有工作流脚本在 worker thread 里。文件地图见 `README 的 Architecture`。

`index.ts` 有 4370 行，是最大的文件，负责注册工具、命令、渲染器和 `/agents` 菜单，默认导出的函数就是扩展入口（`入口`）。注册的 LLM 工具有 `Agent`、`get_subagent_result`、`steer_subagent` 和可选的 `SubagentWorkflow`。

agent 注册表

职责

回答“`subagent_type: "Explore"` 到底是什么配置”。

来源和优先级

来源	位置	代码
内置	`general-purpose`、`Explore`、`Plan`	`default-agents.ts`
项目	`.pi/agents/`、`.agents/agents/`	`custom-agents.ts`
全局	用户目录	`custom-agents.ts`
pi 包	`package.json` 的 `pi.subagents`	`package-resources.ts`

统一入口是 `agent-types.ts`。agent 文件是带 YAML frontmatter 的 Markdown，字段见 `Frontmatter Fields`。

一个重要约定：**frontmatter 是权威的**。文件里写了 `model`、`thinking`、`max_turns`、`isolated` 等，调用方传的值就不生效（`README`）。例外也写明了：`isolation` 在工作流路径上由脚本决定，这是为了和 Claude Code 一致。

类型名大小写不敏感，但如果解析不到恰好一个启用的 agent，会回退或报错，不会悄悄选一个。

包的声明有一个刻意的差异：pi 其他资源会扫描约定目录，子 agent **必须按名字声明**（`packages.md`）。这样装一个包不会意外多出一堆 agent。

> **注意 · 容易误解的地方**
>
> `isolated` 和 `isolation` 是两个字段。`isolated: true` 是“密封模式”：去掉扩展和技能。`isolation: worktree` 是文件系统隔离。名字相近，含义完全不同。

生命周期管理

职责

`AgentManager` 跟踪每个 agent 的记录，管理并发、排队、恢复和清理。`AgentRecord` 是核心状态。

难点：两个并发池

文件头的注释解释了设计理由，这段值得读（`agent-manager.ts`）：

池	默认	管谁
后台	`maxConcurrent = 10`	detached 的 agent
前台	`maxConcurrentForeground = 0`（不限）	调用方阻塞等待的 agent

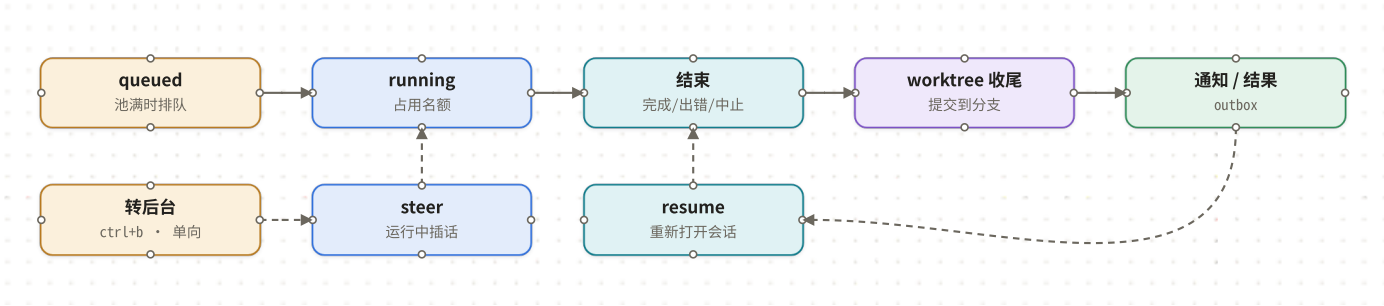
为什么不是一个池？前台 agent 反正会阻塞父 agent，父 agent 本来也能自己干这个活。如果它占后台名额，满载的后台池就会饿死主会话。嵌套子 agent 和工作流 agent 都不占池：嵌套子 agent 排在等它的父 agent 后面会死锁，工作流有自己的上限（`Concurrency`）。

一个精巧的细节

`startAgent` 在**第一个 await 之前**就把状态设成 running 并占用名额。原因：创建 worktree 是异步的，而 `drainQueue` 是同步循环读计数器。如果 await 之后才加一，它会把队列里的 agent 全部启动。

池也只解析一次并记在 `record.chargedPool`：用户可能在运行中改设置，settle 时重新计算会减错池，要么计数下溢，要么名额泄漏。这类注释是这个项目的特点，读它们比读代码本身更省时间。

一个 agent 的状态变化



对应 `spawnAndWait`、`sendToBackground`、`resume`、`steer`。转后台是单向的（`README`）。

被驱逐的 agent 还能恢复：结束时写一条 `subagents:record` 会话条目，`get_subagent_result` 和 resume 都能回退到它（`agent-record-entry.ts`）。

运行器：构造子会话

职责

`runAgent` 把“类型 + prompt + 选项”变成一个跑起来的 `AgentSession`，返回结果。

难点

子会话要和主会话共用 pi 的机制，但配置必须独立。而且默认的 pi 行为会破坏这种独立：`AGENTS.md` 会被自动加到系统提示后面，这样 `prompt_mode: replace` 就失效了。

原理

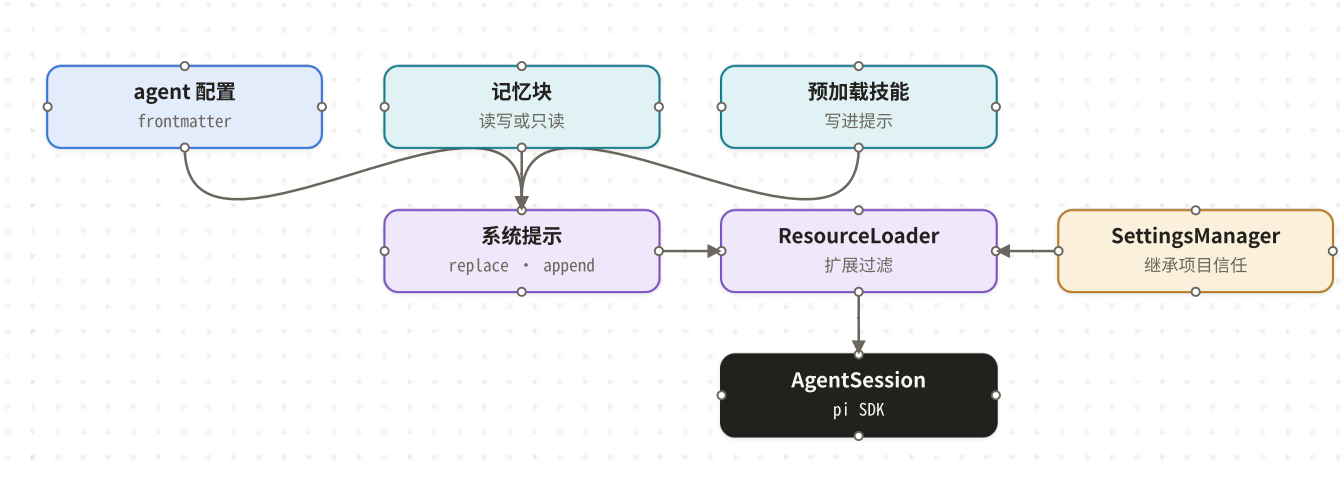
`runAgent` 的主要步骤：

1. 解析工作目录：worktree 覆盖 > 父 cwd。文件操作在 `effectiveCwd`，配置发现在 `configCwd`。
2. `isolated` 在这里解析，不靠调用方，因为 mention 和 RPC 不传这个值。
3. 预加载技能、处理 memory：有写工具就给读写记忆，否则给只读记忆。
4. `prompts.ts` 构建系统提示。
5. 构造 `DefaultResourceLoader`，设 `noContextFiles: true`，手动转发 pi 内置扩展。
6. 继承父项目的信任决定，否则被拒绝信任的项目里，子 agent 会加载 `.pi` 扩展和 MCP 服务。
7. 运行，观察 turn，处理 max turns。

第 6 点是一个安全相关的细节：`SettingsManager` 默认信任，所以必须显式传 `projectTrusted`（代码）。

下面这张图是一个子会话的组成。每一块的来源都不同，这也是 `runAgent` 这么长的原因。

一个子会话由什么组成



系统提示由 `prompts.ts` 构建。`replace` 模式下不继承 `AGENTS.md`，`append` 模式才把父提示嵌进来。

工具和扩展的作用域

这里有三个独立的旋钮，容易混淆：

字段	控制什么	位置
`extensions`	**加载**哪些扩展	`parseExtensionsSpec`
`tools` 里的 `ext:`	扩展工具里**暴露**哪些给模型	`parseExtSelectors`
`disallowed_tools`	无论如何都**拒绝**	`Tool Denylist`

源码注释特别提醒：排除的扩展工厂仍会执行一次，排除只是阻止绑定处理器和注册工具，**它不是沙箱**。

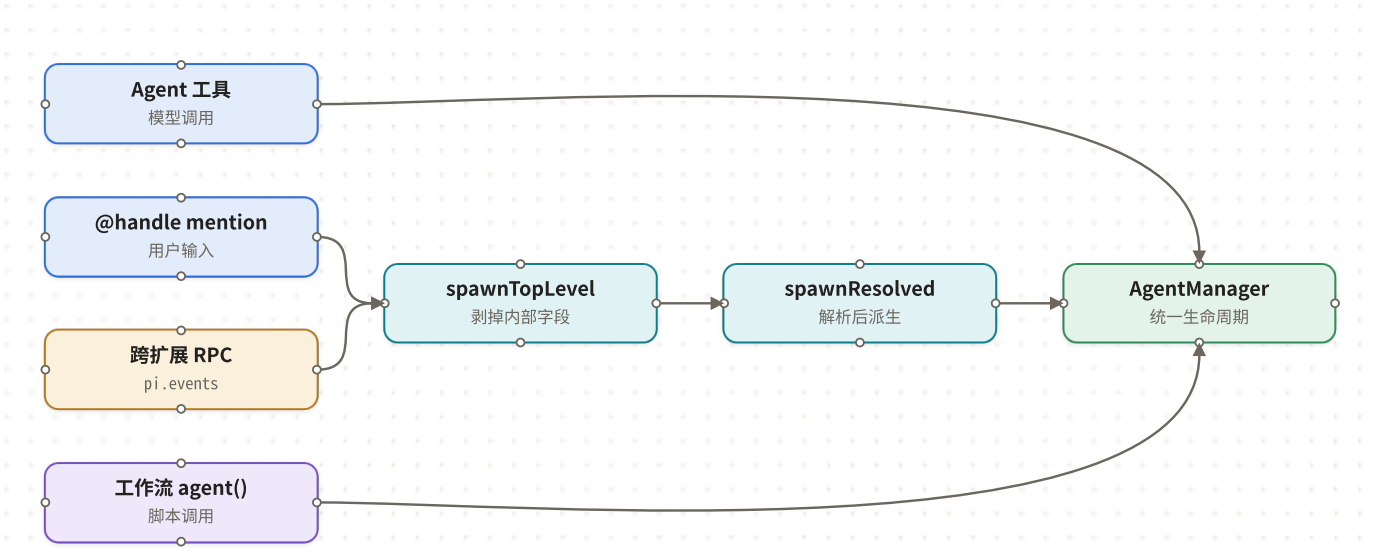
优雅的 turn 上限

硬中止会留下半截输出。`observeRun` 在接近上限时发一条“收尾”提示，再给 `graceTurns` 轮宽限（`Graceful Max Turns`）。这样得到的是完整的部分结果。

入口不止一个

派生 agent 有四条路：`Agent` 工具、`@handle` mention、工作流、跨扩展 RPC。`Agent` 工具和工作流直接调用 manager；mention 和 RPC 的参数不完全可信，要先过一道过滤。

四条派生路径汇到同一处



`Agent` 工具在 `index.ts` 调 `manager.spawn`，工作流在 `host.ts` 调 `spawnAndWait`。`spawnTopLevel` 会删除 `parentAgentId`、`workflowId`、`resumeSessionFile`、`rootSessionId`、`blocking` 等内部字段。

删字段是有安全考虑的：`rootSessionId` 决定转录目录，伪造就是路径穿越；`resumeSessionFile` 决定打开哪个文件当对话重放。注释把每个字段的风险都写清了。唯一的例外是恢复 mention：它走 `spawnResolved`，因为那些值来自扩展自己写的记录

（代码）。

完成通知

难点

后台 agent 完成时，父 agent 可能正在跑工具、空闲，或刚被 Esc 中止。pi 的 steering 队列在默认的 one-at-a-time 模式下会把多条消息拆成多次模型调用，很贵。

原理

`CompletionOutbox` 把通知攒起来，在边界上一次发出：

父 agent 状态	处理
忙	留到 `turn_end`，这时工具批次已结束，下一次模型调用还没开始
空闲	短暂等 `200ms`，让接近同时完成的合并到一轮
被中止或失败	继续持有，等运行结束再发，因为 pi 在 Esc 时会清空自己的队列

另外两个细节：读取结果会取消对应通知（`cancel`），不会既读到又被通知；同一 agent 的新完成替换旧通知。

> **类比 · 前台的信件格**
>
> - **信件** = 子 agent 的完成通知。
> - **你** = 父 agent，开会时不被打断。
> - **信件格** = outbox：会议结束（`turn_end`）一起拿给你。
> - **你已经主动去取了** = `get_subagent_result`：这封信就从格子里撤掉。
>
> 你不在开会时，前台也会等 200ms，看看有没有紧跟着的第二封，好一起送上来。

join 模式在这之上：`smart` 逐个通知；`group` 把同一轮派生的 agent 合成一条，第一个完成后 30 秒超时（`GroupJoinManager`）。

隔离与嵌套

Worktree

`worktree.ts` 创建临时 git worktree。没有改动就清理，有改动就提交到分支，把分支名返回。所有 git 调用都走异步的 `pi.exec`，否则并发隔离的 agent 会卡住 TUI 事件循环。

失败时**不回退**：不是 git 仓库或没有提交时直接报错，绝不悄悄在主树上跑（`startAgent`）。

> **注意 · 容易误解的地方**
>
> worktree agent 结束后改动在分支上，不在主树。所以串行的“修复再验证”不要用隔离，否则验证的是没修过的树。`gated-fix.js` 的文件头专门讲了这个坑。

嵌套

默认关闭。agent 设置 `allowed_subagents` 才会得到作用域内的 `Agent` 等工具（`nested-tools.ts`）。深度上限 `maxSubagentDepth` 默认 2：主会话是 0，子 agent 是 1，再下一层是 2。

工作流

职责

`SubagentWorkflow` 让模型写一段 JavaScript 脚本来编排 agent。**你不写脚本，模型写**（`workflows.md`）。脚本可用 `agent()`、`parallel()`、`pipeline()`、`phase()`、`log()`。

```
export const meta = { name: 'gated-fix', description: '…', phases: [{ title: 'Fix' }] }

phase('Fix')
let fixed = await agent(task, { label: 'fix', gate: testCommand })
if (fixed === null) {
  fixed = await agent(`${testCommand} is still failing…`, { label: 'fix', resume: 'fix' })
}
```

简化自 `gated-fix.js`。

两层边界

这是全项目最有意思的设计，`worker-source.ts` 的文件头讲得很透。

两层边界，目的不同



worker 那层是为了**能杀掉**：`vm` 超时对已经进入 `await` 的脚本无效，`worker.terminate()` 可以。vm 那层是为了**确定性和防手滑**，不是安全边界。`runtime.ts` 说脚本唯一的路径就是 `call` 消息，所以上限才绕不过去。

为什么不算安全边界

文件头直说：注入的全局函数是 host 闭包，`agent.constructor` 还是 host 的 `Function`。真正承重的防线是 `codeGeneration: { strings: false }`，它让 `Function("…")` 和 `eval` 抛错。作者明确写“这是确定性边界，不是针对恶意脚本的安全边界”。

确定性和 journal

工作流可以恢复。恢复靠 `journal` **按前缀重放**：已完成的调用直接返回记录的结果，不重新派生 agent。如果脚本读了时钟或随机数，第二次运行的前缀就对不上，重放会悄悄偏离。

所以 `DETERMINISM_PRELUDE` 把 `Date.now`、`Math.random` 改成抛错，并用词法遮蔽替换 `Date`。错误信息直接告诉作者：时间戳通过 `args` 传入。

prelude 必须写成**一行**：脚本从第 2 行开始编译，host 传 `lineOffset: -1`，多一个换行所有栈帧行号都会错。

▶ 中断后恢复一个工作流 1 / 5



1. 重新运行

带 `resumeFromRunId` 再调用工具，脚本从第一行重新执行。

2. 计算 key

每次 `agent()` 由 `journalKey` 算出 key，所以调用顺序和参数决定匹配。

3. 重放

前缀里的调用立即返回记录的结果，不花 token。

4. 接着跑

第一个没命中的调用开始真正派生 agent。

5. 记录

每个调用完成时追加写入，所以这次运行也能再次恢复。

类比 · 游戏的存档点

- **存档** = journal 里记录的每次 `agent()` 结果。
- **读档** = `resumeFromRunId`：从头跑脚本，但已存档的关卡直接跳过。
- **关卡必须和上次一样** = 确定性：如果地图是随机生成的（`Math.random`），存档就对不上。

禁用时钟和随机数不是洁癖，是读档的前提。

gate

`gate` 是一条 shell 命令，agent 结束后运行，非零退出就算失败，`agent()` 返回 `null`（`applyGate 注释`）。这是“别问模型，跑命令”的落地。worktree agent 的 gate 要在它改过的树上跑，所以交给 host 在 settle 里执行。`gate` 不能和 `resume` 同时用。

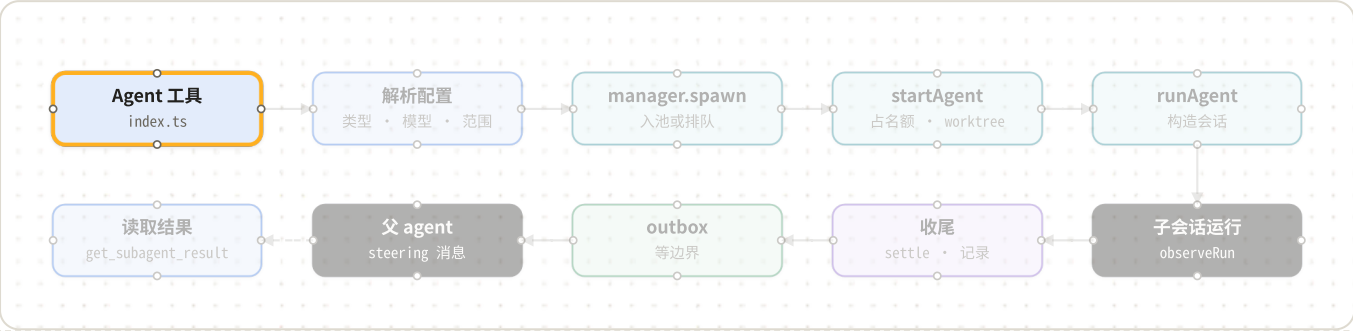
meta

`export const meta` 在 `vm` 里是语法错误，而 UI 又要在执行前知道阶段。`meta.ts` 不引入 acorn，而是自己扫描到匹配的括号，在空 vm 里求值这个片段。纯字面量没有东西可调用，所以是安全的。扫描器要识别字符串、注释和正则，因为 `detail` 文本里经常有花括号。

端到端：一次后台 Agent 调用

假设模型调用 `Agent({ subagent_type: "Explore", prompt: "找认证代码", run_in_background: true })`。

▶ 后台 Agent 调用 1 / 9



1. 工具调用

`Agent` 工具在 `index.ts` 注册。

2. 解析

`resolveAgentInvocationConfig` 合并参数和 frontmatter；`model-resolver.ts` 解析模型；开了 model scope 就检查白名单。

3. 派生

`manager.spawn` 登记记录，池满就排队，工具立即返回 ID。

4. 启动

`startAgent` 同步占用名额，再创建 worktree（如果需要）。

5. 构造会话

`runAgent` 构建提示、loader、设置，创建 `AgentSession`。

6. 运行

子会话跑完，转录同时写到输出文件（`output-file.ts`）。

7. 收尾

释放名额、收尾 worktree、写记录条目，再 `drainQueue`。

8. 投递

通知进 outbox，在父 agent 的下一个边界作为一条 steering 消息送达，带结果预览。

9. 取全文

需要完整输出时调用 `get_subagent_result`，这同时取消尚未送达的通知。

开源许可证

部分	许可证	证据
全部代码	MIT，Copyright 2026 tintinweb	`LICENSE`、`package.json`
运行时依赖	`croner`、`nanoid`	`dependencies`，各自的许可证请自行核对
pi 本体	peer 依赖，不随包发布	`peerDependencies`

MIT 的要求只有一条：分发时保留版权和许可声明。

场景	允许吗	要做什么
自用	允许	无
提 PR	允许	先开 issue；没有看到 CLA 要求
公开 fork	允许	保留 LICENSE
发 npm 包	允许	保留 LICENSE，换包名
对外提供服务	允许	MIT 无网络条款
闭源复用	允许	保留声明

历史：GitHub API 显示 `LICENSE` 只有 2026-03-05 的初始提交一次，一直是 MIT。`media/` 里的 logo 没有单独声明，商用前可以问作者（这是我的提醒，仓库没有写）。

> **不是法律意见**
>
> 这是工程师读代码时的理解。

贡献与定制

开发环境

```
git clone https://github.com/tintinweb/pi-subagents
cd pi-subagents
npm install
npm run check          # lint + typecheck + test
pi -e ./src/index.ts   # 加载本地扩展
```

命令来自 `AGENTS.md` 和 `README`。

项目规则

- **唯一规则：你必须理解自己的代码**，否则 PR 会被关闭（`The One Rule`）。
- 超过小修的改动先开 issue。
- 不要编辑 `CHANGELOG.md`，由维护者写。
- 改用户可见的行为要更新 README。
- 引用代码用符号，不用行号（`文档规则`）。
- 不用内联 import，不用 `any`，top-level imports only。
- PR 标题用 conventional commit，描述分 Summary、What changed、Performance、Testing 等节（`Filing PRs`）。
- 维护者对外部 PR 的审查规则很严：先当作可能恶意来检查。

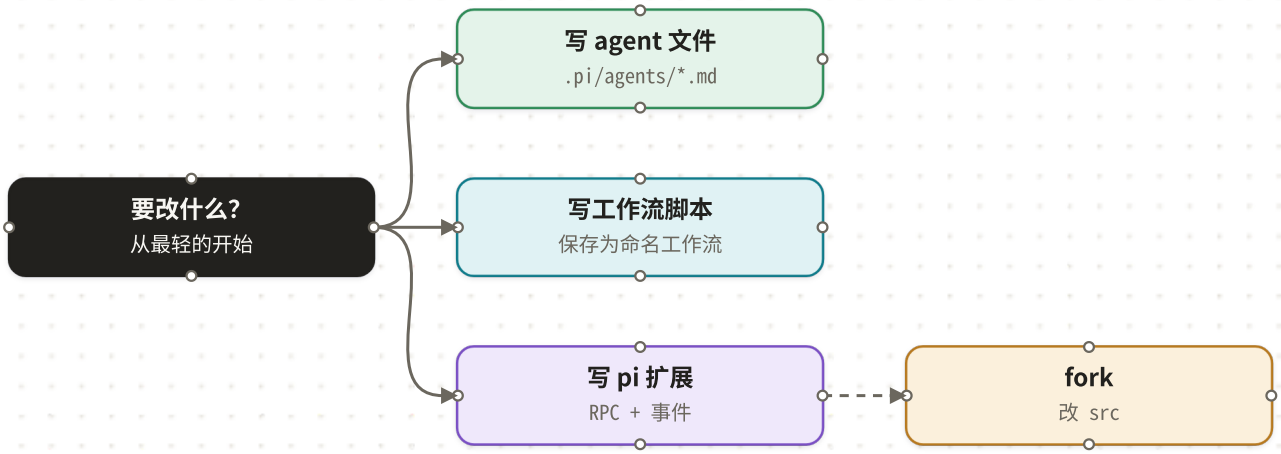
适合入手的方向

- 补 `test/` 里的边界用例，这个项目有 110 多个测试文件。
- `examples/workflows` 加新配方，它们被 `测试` 执行。
- 改善文档。

需要避开：和 Claude Code 不一致的工具名和参数；范围之外的新功能。

定制路线

想让它按我的方式工作



其他扩展通过进程内事件总线调用，见 `rpc.md` 和 `cross-extension-rpc.ts`。发包见 `packages.md`。

路线	代价	适合
agent 文件	很低	专用角色、模型、工具集
工作流	低	可复用的编排流程
发 pi 包	低	团队共享 agent 和工作流
写扩展调 RPC	中	和其他功能集成
fork	高	改核心行为，要跟随 pi 的 API 变化

仓库里没有的东西

- **pi 本体**：`@earendil-works/pi-coding-agent`、`pi-ai`、`pi-tui` 都是外部包。`AgentSession` 的内部行为要看它们的源码。
- **沙箱**：子 agent 拥有用户权限。需要隔离就放进容器或 VM，SECURITY.md 说得很明确。
- **防提示注入**：SECURITY.md 明说 `AGENTS.md`、agent 文件都可以注入，且无法防御。只在可信仓库里使用。

附录：源码索引

主题	链接
入口	`index.ts`
类型	`types.ts`
生命周期	`agent-manager.ts`
运行器	`agent-runner.ts`
通知	`completion-outbox.ts`、`group-join.ts`
注册表	`agent-types.ts`、`custom-agents.ts`
隔离	`worktree.ts`、`nested-tools.ts`
调度	`schedule.ts`、`schedule-store.ts`
记忆	`memory.ts`
工作流	`runtime.ts`、`worker-source.ts`、`meta.ts`、`journal.ts`
UI	`agent-widget.ts`、`fleet-list.ts`、`conversation-viewer.ts`
设置	`settings.ts`
文档	`workflows.md`、`rpc.md`、`packages.md`
规则	`AGENTS.md`、`CONTRIBUTING.md`、`SECURITY.md`